

Le design pattern Strategy

Marc Champesme

`mailto:Marc.Champesme@univ-paris13.fr`

18 février 2026

- Comme le design pattern Iterator il appartient à la catégorie des patterns comportementaux
- il est utile lorsque plusieurs versions d'un algorithme peuvent être utilisées pour une fonctionnalité particulière d'une classe
- il s'agit alors d'encapsuler chaque algorithme dans une classe (cf. `Comparator<E>`).
- le changement d'algorithme se fait alors en fournissant à la classe une instance de la classe correspondant à l'algorithme choisi

Références biblio dans [Bloch 2018] : Chapitre 7 "Lambdas and Streams"

Exemple en Java : l'interface Comparator (1)

Définition de l'interface Comparator :

```
public interface Comparator<E> {  
    /** Compares its two arguments for order.  
     * Returns a negative integer, zero, or a  
     * positive integer as the first argument is  
     * less than, equal to, or greater than the second.  
     */  
  
    int compare(E o1, E o2);  
}
```

Utilisation dans l'interface List avec la méthode sort :

```
/** Sorts this list according to the order  
 * induced by the specified Comparator. */  
  
void sort(Comparator<? super E> c)
```

Utilisation d'un Comparator

Si je veux trier une liste d'ObjetZork en fonction de leurs poids :

```
public class CompareWeight
    implements Comparator<ObjetZork> {
    public int compare(ObjetZork o1, ObjetZork o2) {
        return o1.getWeight() - o2.getWeight();
    }
}
```

Utilisation avec une liste :

```
List<ObjetZork> loz = ...
loz.sort(new CompareWeight());
// liste loz triee selon le poids des ObjetZork !
```

Utilisation d'un Comparator (suite)

Et si je veux trier une liste d'ObjetZork en fonction de leurs noms ?

```
public class CompareName
    implements Comparator<ObjetZork> {
    public int compare(ObjetZork o1, ObjetZork o2) {
        return o1.getName().compareTo(o2.getName());
    }
}
```

Utilisation avec une liste :

```
List<ObjetZork> loz = ...
loz.sort(new CompareName());
// liste loz triee selon le nom des ObjetZork !
```

En pratique, nous verrons plus tard que (en Java) on peut obtenir le même résultat sans définir les classes CompareWeight ou CompareName.

Autre exemple : Application au jeu Zork

Nous voulons ajouter une méthode à la classe `Gamer` qui fait une recommandation pour le choix du prochain déplacement (choix parmi les pièces voisines de la localisation actuelle du joueur) :

```
public class Gamer {  
    // ...  
    public Location getRecommandation() {  
        // Calcul le meilleur choix parmi les pieces  
        // voisines de la position actuelle du joueur  
    }  
}
```

Plusieurs stratégies sont possibles pour ce choix : recherche de nourriture, d'armes, de soins, d'un trésor, éviter des endroits dangereux, ...

Comment prendre en compte différentes stratégies ?

Première solution :

- une interface `Gamer`
- une classe abstraite `AbstractGamer` dont la seule méthode abstraite serait la méthode `getRecommandation`
- une sous-classe de `AbstractGamer` pour chaque stratégie

Et on obtient une multitude de classes héritant de `Gamer` dont la seule différence est l'implémentation de la méthode `getRecommandation`.

Par ailleurs, la stratégie du joueur est fixée une fois pour toute et le joueur ne peut pas changer de stratégie au cours du jeu.

Au premier cours nous avons énoncé un principe :

Principe :

Distinguer les parties du code qui sont susceptibles de changer et les isoler dans des classes spécifiques : minimiser la quantité de code à modifier, localiser le code à modifier dans un petit nombre de classes.

On va donc isoler le code qui change dans des classes qui ne contiennent que le code de la stratégie.

Autre solution avec le pattern Strategy

On définit une interface MovingStrategy :

```
public interface MovingStrategy {  
    Location getLocation();  
}
```

On définit une classe implémentant cette interface pour chaque stratégie. Puis on ajoute un attribut de type MovingStrategy dans la classe Gamer :

```
public class Gamer {  
    private MovingStrategy myStrategy;  
    //...  
    public Location getRecommandation() {  
        return myStrategy.getLocation();  
    }  
}
```

Reste à choisir la stratégie...

À la création :

```
public class Gamer {  
    private MovingStrategy myStrategy;  
    public Gamer(MovingStrategy strtgy) {  
        myStrategy = strtgy  
    }  
    //...  
    public Location getRecommandation() {  
        return myStrategy.getLocation();  
    }  
}
```

Choix de la stratégie (1)

Ou bien pendant le jeu, puisque la stratégie est un attribut dont la valeur peut être modifiée :

```
public class Gamer {  
    private MovingStrategy myStrategy;  
    // Autres attributs...  
    public Gamer(MovingStrategy strtgy) {  
        myStrategy = strtgy  
    }  
    // Autres methodes...  
    public void eat() {  
        // consomme des aliments  
        if (!hasFood()) {  
            myStrategy = new FoodStrategy(zorkMap, this);  
        }  
    }  
}
```

Choix de la stratégie (2)

Ou peut aussi utiliser ces stratégies dans d'autres classes que Gamer :

```
public class Monster {  
    private ZorkMap map;  
    private MovingStrategy myStrategy;  
    // Autres attributs...  
    public Monster(ZorkMap map) {  
        this.map = map;  
        myStrategy = new FightStrategy(map, this);  
    }  
    // Autres methodes...  
    public void move() {  
        map.setLocation(this, myStrategy.getLocation());  
    }  
}
```

Le pattern Strategy en Java

Le pattern Strategy est très utilisé dans la bibliothèque de classes Java, particulièrement depuis la version 1.8 du langage et de la bibliothèque de classe (exemples ici pour l'interface List) :

`boolean removeIf(Predicate<? super E> filter)` Removes all of the elements of this collection that satisfy the given predicate.

`T[] toArray(IntFunction<T[]> generator)` Returns an array containing all of the elements in this collection, using the provided generator function to allocate the returned array.

`void replaceAll(UnaryOperator<E> operator)` Replaces each element of this list with the result of applying the operator to that element.

Le pattern Strategy en Java

Predicate, IntFunction et UnaryOperator sont appelées *functional interface*. Une *functional interface* est définie dans la spécification du langage par :

A functional interface is an interface that [...] has just one abstract method (aside from the methods of Object), and thus represents a single function contract.

Quelques unes des nombreuses *functional interface* définies dans le package `java.lang.function` :

Interface	Unique méthode définie
Predicate<T>	<code>boolean test(T t)</code>
Function<T,R>	<code>R apply(T t)</code>
UnaryOperator<T>	<code>T apply(T t)</code>
IntFunction<R>	<code>R apply(int value)</code>
Supplier<T>	<code>T get()</code>

Les *functional interfaces* en Java

Un défaut des *functional interfaces* est qu'il peut conduire à la création de nombreuses classes dont le code est très simple (cf. cas de `Comparator`) : création d'un nouveau fichier à chaque fois contenant très peu de code *utile*. Pour éviter cela, la technique utilisée en Java était de créer des classes *anonymes*.

Par exemple, au lieu de créer une classe `CompareName`, on peut utiliser le code suivant :

```
Comparator<ObjetZork> cmp
= new Comparator<ObjetZork> {
    @Override
    public int compare(ObjetZork o1, ObjetZork o2) {
        return o1.getWeight() - o2.getWeight();
    }
};
```

Cela fait toujours beaucoup de caractères pour pas grand chose...

Les *lambda expressions* en Java

Pour alléger encore l'écriture du code et améliorer sa lisibilité, les concepteurs de Java ont introduit les *lambda expressions* depuis la version 1.8 du langage.

Ce qui nous permet d'écrire maintenant :

```
Comparator<ObjetZork> cmp;  
cmp = (o1, o2) -> o1.getWeight() - o2.getWeight();
```

Attention !!!

L'usage des *lambda expressions* est réservé aux *functional interfaces* : les interfaces qui ne possèdent qu'une seule méthode. Et l'usage des *lambda expressions* doit être réservé aux cas où le code est très simple (une expression simple qui tient sur une ligne).